Correct but Not Parallel: A Grounded Agentic Critic for LLM-Generated HPC Code

Anonymous Author(s)
Affiliation withheld for review
Location withheld
email withheld

***Abstract*—Agentic systems now write HPC code and decide for themselves when it is done: the agent generates an OpenMP kernel, runs it, compares the output against a trusted serial reference, and accepts the draft if they agree. That acceptance test is the agent's objective—and, increasingly, the verifiable reward a post-training loop optimizes. We show it is the wrong one, and we build the agent that sees why.**

Our contribution is a *grounded agentic diagnostician* for parallel code: an LLM that answers questions about a generated program only by calling real measurement tools—compile, sweep thread counts, time against the serial reference, check for races—and a *performance-critic* agent that refuses any claim a measurement does not support, the way a grounded explanation refuses an attribution it cannot compute. Building it forced the finding that motivates it. Delete every `#pragma omp` from a correct program and it stays correct, so the serial program sits in the argmax of the acceptance reward and the reward is invariant to the one property the code exists for; we make this executable, and 96.8% of the programs our verifier accepted survive mechanical pragma-stripping at exactly zero reward cost. The blind spot is large: across 63 accepted programs, all scored identically correct, measured 8-thread self-speedup runs from 0.33× to 7.91×, and 1.6% are *slower* than the serial code they replace. On saturated correctness (82.8% single-shot) a group-relative estimator sees zero reward variance on a predicted 88.8% of groups and learns nothing. We release PARALLELGATE, an adversarial suite of 48 candidates encoding the known ways to defeat a parallel-code reward, and score six reward formulations against it: a multiplicative efficiency gate $R_{\text{gate}} = \mathbb{1}[\text{correct}] \cdot \min(S_n/n, 1)$ resists every reward-*shape* attack and is defeated only by attacks that corrupt the measurement itself, which no reward shape can fix. The diagnostician is exactly a deployable form of that gate: a critic that grounds every parallelism claim in a measurement, and reports an honest null when it cannot.

***Index Terms*—agentic AI, HPC, code generation, OpenMP, grounded diagnosis, verifiable rewards, RLVR, reward hacking, strong scaling, evaluation, reliability**

I. INTRODUCTION

The checker became the objective. Code models are increasingly post-trained not on human preference but on execution: the model writes a program, an automatic checker runs it, and the reward is paid only if the program does what it was asked to do [1]–[3]. The same substitution has happened inside agents, which decide for themselves that a draft is finished by running it and looking at the output. In both settings the checker has moved out of test infrastructure and into the objective function. Whatever it scores highly is what gets written.

For parallel code the checker is the one everyone already has: compile the generated program, run it, compare its output against a trusted serial reference, pay the reward if they agree. This is what the standard benchmark for LLM-generated parallel code does [4], what an execute-and-fix agent does when it accepts its own OpenMP draft [5], [6], and the natural verifiable reward to reach for when post-training an HPC code model. It is also the wrong objective-not for want of tuning, but structurally. We found this by trying to break our own verifier.

Take a correct OpenMP program and delete every `#pragma omp` line in it. It still compiles. It still runs. It still agrees with the serial reference on every input at every thread count-more reliably than before, in fact, because there is now nothing left to race. So it still earns the maximum reward. The serial program is in the argmax. This is not a corner case or a weak spot to be patched: the reward is a predicate on outputs, the transform preserves outputs by construction, and so no amount of additional testing removes it. A reward for parallel code that is maximized by code with the parallelism deleted is not measuring parallel code.

Two consequences follow, and they differ in kind. The first concerns the *optimum*. No procedure that maximizes this reward has any incentive to emit parallel code. Where models do write parallel code under such an objective they are drawing on the pretrained prior, not responding to the reward; training preserves that behaviour only by accident, and can erode it at no cost. For code whose entire reason to exist is to run faster on more cores, an objective indifferent to whether it does is not a small mismatch.

The second concerns the *gradient*, and it bites long before the optimum is approached. Group-relative estimators such as GRPO center each sample's reward on its group mean and divide by the group standard deviation. Correctness on current parallel-code benchmarks is close to saturated-82.8% of our single-shot generations are already correct at every thread count we test. When a sampled group is uniformly correct the reward is uniformly 1, the standard deviation is zero, and the advantage is zero or undefined. The reward supplies no signal, not because the model is right, but because the reward has

nothing further to say about it. Sharper correctness checking makes this worse, not better: it is the one axis on which there is nothing left to win.

What we measure: We turn both claims into measurements on a frozen pool of 63 verified programs, so that every number in this paper can be recomputed offline without an inference budget. First, we make the degeneracy executable: we implement the serial projection π_{serial} as a mechanical source transform that strips every OpenMP directive, and 96.8% of projected programs still pass the correctness harness, at a mean reward change of exactly 0.00 under the correctness reward-not an argument that serial programs are in the argmax, but 63 instances of it. Second, we measure what the reward is blind to: 8-thread self-speedup across the accepted pool runs from 0.33× to 7.91×, and 1.6% of accepted programs are slower on eight threads than on one-correct, certified, and worse than the serial code they were meant to replace, every one of them receiving reward 1. Third, we measure the signal collapse directly, as the fraction of sampled groups carrying zero within-group reward variance: 88.8% at $k = 8$ under the correctness reward, against 12.5% under a reward with a continuous performance term. Fourth, we treat reward design as something to be tested rather than proposed, releasing PARALLELGATE, a suite of 48 adversarial candidates encoding the known ways to defeat a parallel-code reward-pragmas deleted, `num_threads(1)`, whole-body `critical`, a deliberately slowed single-thread baseline, algorithmic speedup without threading, work moved outside the timed region, and cross-run caching-and scoring six reward formulations against all of them. The efficiency-gated reward $R_{\text{gate}} = \mathbb{1}[\text{correct}] \cdot \min(S_n/n, 1)$ resists most of these; we report, without softening it, the classes where it does not.

What we do not claim: The literature has moved quickly and we want the boundary of the contribution to be exact. We are *not* the first to observe that model-generated parallel code can be functionally correct while executing serially: PARACODEX [7] documents this as "bypass" and "pseudo-offload" in a GPU setting and characterizes it as an agent and harness failure to be patched with harness constraints. Our difference is what the observation is a fact *about*-not a bug in a particular harness, but the location of the optimum of the reward function itself, on CPU thread scaling rather than device offload. We are *not* the first to combine correctness with measured performance in a reward for code: ACECode [8] does so additively for Python runtime; Kevin [9], CUDA-L1 [10], CUDAPerf [11] and MaxCode [12] do so for GPU kernels; RLPF [13] aligns code models to performance through a learned reward model and reports 1.9–4.5× on OpenMP; and real-machine GFLOPS rewards have been used to post-train an HPC code model with GRPO [14]. Our contribution is not the combination, but the argument for why the correctness term cannot stand alone, the structure that follows from it-a multiplicative gate rather than a weighted sum-and an adversarial suite that lets these formulations be compared rather than merely listed. Finally, we are *not* claiming that best-of-N selection is equivalent to group-relative policy optimization: it is a proxy for the selection component only [15], since GRPO additionally sharpens the sampling distribution [16], and we label our selection analysis as such throughout.

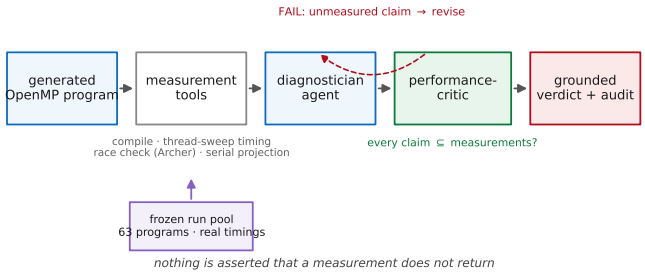

Fig. 1. The grounded agentic diagnostician. The agent may call only measurement tools—compile, thread-count sweep timing, an independent race check, and the serial projection—over a frozen pool of executed programs; the performance-critic fails any drafted claim not entailed by those measurements and returns it for revision. The same efficiency gate the critic enforces at inference (Section IV) is the reward a post-training loop should optimize—the checker and the reward are one object.

Contributions:

1) **Optimum degeneracy.** A statement and proof, via the serial projection, that correctness-only verifiable rewards for parallel code are maximized by programs with no parallelism, with the corollary that group-relative estimators suffer *signal collapse* on saturated correctness distributions (Section III).
2) **An executable demonstration.** Pragma-stripping the accepted pool preserves correctness for 96.8% of programs at zero reward cost, turning the argument into instances (Section VI-A).
3) **A quantification of the blind spot.** Accepted programs range from below-serial to near-linear scaling at identical reward, and correctness-only selection discards 15.0% of the available speedup (Sections VI-B, VI-D).
4) **PARALLELGATE.** An adversarial reward-hack suite for parallel code, and a bake-off of six reward formulations against it-including the cases our own gate fails (Section VI-E).
5) **A measurement protocol and a released artifact.** A timing protocol that closes the measurement-corruption attacks (fresh processes, whole-driver timing, randomized inputs), together with an artifact that recomputes every reward and figure in the paper offline (Section VI-F).

II. A GROUNDED AGENTIC DIAGNOSTICIAN

The object we build is an agent that answers questions about a generated parallel program the way a grounded model-explanation answers questions about a prediction: by calling tools that return real measurements, and by saying nothing the tools did not return. It has two roles, and a discipline that binds both (Figure 1).

Tools, not assertions: The diagnostician is a tool-calling LLM over a fixed set of measurement tools, each of which reports a fact about a program that was actually compiled

and run: its correctness at every thread count in the sweep $\{1, 2, 4, 8\}$; its per-thread wall-times and the resulting self-speedup $S(p) = T(1)/T(p)$ and efficiency $S(p)/p$; its speedup against the trusted serial reference; the race verdict from an independent happens-before detector; the synchronization idiom it used; and the *serial projection*—the source with every OpenMP directive deleted—together with the reward each formulation pays it. The agent chooses which tools to call and narrates their results; it is forbidden, by system prompt and by the critic below, from stating a speedup, a race verdict, or a "correct but not parallel" judgement that no tool returned. This is the parallel-code analogue of never inventing a feature attribution: a diagnosis is admissible only if a measurement backs every number in it.

The performance-critic: A second agent re-reads the tool outputs—the only source of truth—and the draft diagnosis, and fails any claim the measurements do not support: an invented speedup, a race the detector did not find, or an assertion of scaling where none was measured. It is the acceptance gate the naive agent lacks. A correctness-only self-acceptance loop terminates when the output matches the reference, which (Section III) it does for programs that are correct and serial; the performance-critic terminates only when the measured efficiency, not just the output, is what it claims. The rest of this paper is the argument for why that second gate is necessary and how it should be shaped: the degeneracy of the output-only gate (Section III), the efficiency-gated reward the critic enforces (Section IV), and its behaviour under adversarial attack (Section VI-E).

A diagnosis, grounded: Asked for the worst-scaling accepted program, the agent calls the read tool and reports: task `histogram_bin_0-100`, correct at every thread count and race-clean, yet self-speedup *0.33*× at 8 threads (efficiency 0.04)—slower than serial. Correctness-only reward: *1.0*, identical to a program that scales to 7.91×; efficiency-gated reward: *0.04*. Two programs the verifier certified equally correct on this one task span 0.33–6.82×. Every number in that paragraph is a tool return, and the critic passes it because every number is; strip the measurements and the agent has nothing to say, which is the point. The remainder of the paper measures, on a frozen pool of 63 programs, exactly what that grounding buys.

III. The Degeneracy

This section states the paper's central claim precisely enough to be checked. The argument is short. Its consequences are not.

One object plays two roles throughout, and keeping them straight removes most of the apparent complexity. The correctness check c is what an agent runs to accept its own draft *and* the verifiable reward a post-training loop maximizes—the same predicate wearing two hats. The optimum-degeneracy result (Proposition 1) indicts both at once: an agent accepts correct-but-serial code, and a training optimum sits on it. The signal-collapse result (Proposition 2) is specific to the training loop. The efficiency-gated reward of Section IV is, correspondingly, both the acceptance gate our critic enforces at inference and the reward we recommend for post-training. Where a claim applies to only one role we say so.

A. Setup and notation

Let $\mathcal{T}$ be a set of parallel-programming tasks. Each task $t \in \mathcal{T}$ is given by a natural-language prompt with an explicit function contract, and is equipped with a trusted serial reference R_t: a sequential implementation that defines the correct output on any input. A policy π samples a program $p \sim \pi(\cdot \mid t)$.

A parallel program admits a family of execution configurations. We fix attention on the OpenMP thread count n and write $c(p) \in \{0, 1\}$ for the correctness indicator: $c(p) = 1$ iff p compiles, runs, and matches R_t on every validation input at every thread count and repetition tested. This is the strongest output-based check in common use—strictly stronger than a single execution, which is the usual practice [4]—and the argument below applies to it, and therefore a fortiori to weaker checks.

For a program that compiles and validates, let $T_p(n)$ be its wall time on n threads and T_{R_t} the wall time of the serial reference. We use three performance quantities:

$$S_p(n) = T_{R_t}/T_p(n) \quad \text{(reference speedup)} \tag{1}$$

$$\Sigma_p(n) = T_p(1)/T_p(n) \quad \text{(self-speedup, strong scaling)} \tag{2}$$

$$Q_p = T_{R_t}/T_p(1) \quad \text{(single-thread quality)} \tag{3}$$

which satisfy the identity

$$S_p(n) = Q_p \cdot \Sigma_p(n). \tag{4}$$

Equation (4) separates speedup obtained by writing a better sequential algorithm from speedup obtained by using more cores. It will matter twice: once because a reward that ignores it is gameable, and once because a reward that uses only Σ is gameable differently.

The correctness-only verifiable reward is

$$R_{\mathrm{corr}}(p) = c(p). \tag{5}$$

B. The serial projection

Definition 1 (Serial projection)**.** The serial projection π_{serial} maps a program p to the program obtained by deleting every OpenMP directive from its source: every line beginning `#pragma omp`, and every OpenMP runtime call, replaced by its single-threaded semantics.

π_{serial} is a purely syntactic transform. It requires no understanding of the program. It is implementable in a few dozen lines, and we release ours.

Proposition 1 (Optimum degeneracy)**.** *Let p be a program with $c(p) = 1$ whose parallel regions are free of side effects outside the memory the corresponding sequential loop would touch. Then $c(\pi_{\mathrm{serial}}(p)) = 1$, and therefore*

$$R_{\mathrm{corr}}(\pi_{\mathrm{serial}}(p)) = R_{\mathrm{corr}}(p) = \max_{p'} R_{\mathrm{corr}}(p').$$

Consequently $\pi_{\text{serial}}(p) \in \arg\max R_{\text{corr}}$*, and* $\Sigma_{\pi_{\text{serial}}(p)}(n) \leq 1$ *for all* n*.*

Argument. OpenMP worksharing constructs are, by specification, semantically transparent in the absence of data races: executing a `parallel for` region with one thread is equivalent to executing the loop body sequentially in iteration order. Removing the directive therefore yields the sequential execution that c compares against R_t. If p was correct at $n = 1$—which $c(p) = 1$ requires—then $\pi_{\text{serial}}(p)$ produces that same output at every n, because it has no parallelism whose schedule could vary. Hence $c(\pi_{\text{serial}}(p)) = 1$. Since R_{corr} takes values in $\{0, 1\}$ and $R_{\text{corr}}(p) = 1$ is maximal, $\pi_{\text{serial}}(p)$ attains the maximum. It contains no parallel regions, so it cannot exceed unit self-speedup. □

The proposition is elementary, and we do not present it as difficult. We present it because the field is currently deploying R_{corr} as a training objective for exactly this task, and because it has a consequence that is not elementary: *the correctness reward can be maximized without ever producing a parallel program, so no procedure that maximizes it can be said to have learned parallelism.* Any parallelism observed under such an objective is inherited from the pretrained prior. The reward neither reinforces nor protects it.

Note also what the proposition does *not* depend on. It does not depend on the checker being weak. Strengthening c—more inputs, more thread counts, more repetitions, a happens-before race detector alongside—raises the bar for p and leaves $\pi_{\text{serial}}(p)$ untouched, because a program with no concurrency passes every concurrency check trivially. Improving the correctness verifier moves the reward's argmax *toward* the serial program, not away from it. This is the sense in which the degeneracy is structural.

C. Signal collapse

Proposition 1 concerns where the optimum lies. The second consequence concerns whether any gradient points anywhere at all.

Group-relative policy optimization and its variants estimate an advantage for each sample in a group of size k by centering on the group mean and normalizing by the group standard deviation [3], [17]–[20]:

$$\hat{A}_i = \frac{r_i - \mu(\mathbf{r})}{\varsigma(\mathbf{r})}, \qquad \mathbf{r} = (r_1, \ldots, r_k). \tag{6}$$

Proposition 2 (Signal collapse)**.** *Let* p_t *be the per-task probability that a sample is correct. Under* R_{corr}*, the probability that a group of* k *i.i.d. samples for task* t *yields* $\varsigma(\mathbf{r}) = 0$*—and hence a zero or undefined advantage for every member—is*

$$\Pr[\textit{dead group}] = p_t^k + (1 - p_t)^k. \tag{7}$$

This quantity approaches 1 *as* $p_t \to 1$*. Under any reward of the form* $\mathbb{1}[c] \cdot f$ *with* f *continuously distributed over correct programs, the same probability is at most* $(1 - p_t)^k$*.*

Argument. R_{corr} takes two values, so the group reward vector has zero variance exactly when all k samples are correct or all k are incorrect, giving (7). Under $\mathbb{1}[c] \cdot f$, an all-correct group has zero variance only if all k measured f values coincide, which has probability zero for a continuous f; the all-incorrect case is unchanged. □

The practical reading of Proposition 2 is uncomfortable. The regime in which a correctness reward stops teaching anything is precisely the regime of a *capable* model on a *well-specified* benchmark. Effort spent making models better at correctness, or benchmarks cleaner, drives p_t up and drives the usable signal down. DAPO's dynamic sampling [17] discards zero-variance groups explicitly, which is the correct engineering response and also an admission of the problem: on a saturated task distribution, most of the sampling budget is discarded.

D. Correctness is portable; performance is not

Observation 1 (Machine invariance)**.** R_{corr} *is invariant to thread count, loop schedule, thread affinity, compiler, and target architecture.* $S_p(n)$ *and* $\Sigma_p(n)$ *are not.*

For most software this is a virtue: a correctness oracle that changed with the machine would be a bad oracle. For HPC it identifies the mismatch. The purpose of writing OpenMP rather than a sequential loop is to exploit a particular machine. A reward that is constant across every machine therefore defines an optimum that is, by construction, independent of the only thing the task is about—and an optimum that is tied to whatever machine the timing was collected on, which is why we measure on a fixed machine and treat cross-machine transfer as an explicit limitation (Section VII).

E. Why more verification does not fix it

It is tempting to read Proposition 1 as a call for a better verifier, and we spent some time making that mistake. The relevant distinction is between *extensional* verification—does the program produce the right outputs—and *intensional* verification—does it produce them the right way [21]. Differential testing across thread counts, repetition to expose nondeterminism, and happens-before race detection [22], [23] are all improvements to the extensional check. They tighten the set of accepted programs. They do not remove the serial program from it, because the serial program is extensionally perfect.

What is needed instead is a second observable that is not a function of the output. Wall time is the obvious one, it is what the HPC community already measures, and it is the one we adopt in Section IV. Its cost is that it is a physical measurement rather than a logical predicate, with all the reproducibility difficulty that implies—which is why Section VI-F treats measurement reliability as a first-class result rather than an appendix.

IV. An Efficiency-Gated Verifiable Reward

Section III says a correctness-only reward cannot work. It does not say what should replace it, and the space of replacements is not empty—several are already published. This section states the one the degeneracy argument implies, explains why its *structure* rather than its ingredients is the

claim, and describes the suite we built to test it against the alternatives.

A. The gate

For a program p with correctness indicator $c(p)$ and reference speedup $S_p(n)$ measured on n threads, define the capped strong-scaling efficiency

$$E_p(n) = \min\left(\frac{S_p(n)}{n},\ 1\right) \tag{8}$$

and the efficiency-gated reward

$$R_{\text{gate}}(p) = c(p) \cdot E_p(n). \tag{9}$$

Equation (8) is not new and we do not present it as new: it is the strong-scaling efficiency the HPC community has used for decades, and in the LLM setting it is essentially PAREVAL's own $\text{efficiency}_n@k$ metric [4] with a cap at unity. Naming that lineage matters, because the point of the paper is not the metric but where it belongs—in the reward, not only in the evaluation table.

Under R_{gate}, an incorrect program scores 0; the serial projection of Definition 1 scores approximately Q_p/n, which for a program of ordinary sequential quality is $1/n$—0.125 at eight threads; and a program achieving linear scaling scores 1. Proposition 1 no longer holds: $\pi_{\text{serial}}(p)$ is now separated from p by $E_p(n) - 1/n$, which is the entire dynamic range of the reward.

B. Why a gate and not a weighted sum

The published correctness-plus-performance rewards for code are, with consistency, *additive*. ACECode [8] awards 0.5 for passing tests and adds up to 0.5 of an efficiency term. Kevin [9] awards a correctness constant plus a speedup ratio. Real-machine HPC post-training [14] uses a staged penalty ladder terminating in a throughput term. Each is a reasonable engineering choice and each was validated in its own setting.

The degeneracy argument implies a different structure, for two reasons.

First, an additive reward has a *floor* that correctness alone reaches. A program that is correct and entirely serial banks the correctness constant unconditionally. The optimum is no longer serial, but the serial program is no longer near-zero either; it is a safe, defensible answer that a risk-averse policy can retreat to whenever parallelization is hard. A multiplicative gate makes the performance term unreachable without correctness *and* makes correctness worth nothing without performance. There is no floor to retreat to.

Second, and more practically, the gate preserves the property that made verifiable rewards attractive in the first place: chasing reward can never buy a wrong answer. Under an additive reward with a large enough efficiency coefficient, a fast incorrect program can outscore a slow correct one, and whether it does is a matter of hyperparameter tuning. Under Equation (9) it cannot, ever, at any coefficient.

We state this as a structural argument, not an empirical superiority claim. Section VI-D measures the difference; we did not want the comparison to rest on the argument alone.

C. What the gate does not fix

Adding a measured quantity to a reward adds a measurement to attack. We name the attacks we know of, and we build them (Section VI-E) rather than only discussing them.

Baseline manipulation: If efficiency is computed from self-speedup $\Sigma_p(n) = T_p(1)/T_p(n)$, the policy can raise its reward by making the one-thread run slower. This is the same family as CUDA-L1's timing exploits, where added asynchronous streams misled a main-stream timer [10]. Anchoring to the fixed reference, $S_p(n) = T_{R_t}/T_p(n)$, removes this: the denominator is not under the policy's control.

Algorithmic speedup in parallel clothing: Anchoring to the reference introduces the dual problem. By Equation (4), $S_p(n) = Q_p\Sigma_p(n)$, so a program that improves the sequential algorithm by a factor of n and uses no threads at all reaches $E_p(n) = 1$. The cap conceals it. We therefore report Q_p and $\Sigma_p(n)$ separately throughout, and recommend that any deployed reward do the same; a reward that reports only $S_p(n)$ cannot distinguish a good parallel program from a good sequential one.

Timing-region evasion and caching: Work moved outside the timed region, lazily evaluated, or memoized across the repeat runs used for noise reduction will all inflate the efficiency term. The mitigations are ordinary and unglamorous—time the whole driver, randomize inputs per repeat, run repeats in fresh processes—and we specify them in Section V-D.

The measurement itself: Unlike $c(p)$, $E_p(n)$ is a physical measurement with a noise distribution. A reward whose between-program differences are smaller than its within-program variance is not a reward, it is noise with a mean. Our protocol (Section V-D) times in fresh processes and reports the minimum over repeats to suppress this; quantifying a per-machine noise floor and the resulting reward resolution is a measurement we recommend any deployed performance reward report.

D. PARALLELGATE: an adversarial suite for parallel-code rewards

A reward for parallel code should be judged the way a verifier is judged: by what it accepts that it should not. We therefore built PARALLELGATE, a suite of 48 hand-written candidates spanning PAREVAL's OpenMP problem families, each of which is *correct* and each of which defeats at least one natural reward. The classes are listed in Table I.

The suite is the reusable part of this paper. A reward formulation can be scored against it in minutes, and the resulting matrix (Figure 5, Section VI-E) is a far more informative summary of a reward than the single benchmark number usually reported. We release it with the intention that subsequent reward designs for HPC code report against it, including designs that beat ours.

V. EXPERIMENTAL SETUP

A. Tasks

We build on PAREVAL [4], the standard benchmark for LLM-generated parallel code, which comprises 420 problems

TABLE I
THE PARALLELGATE HACK CLASSES. EVERY CANDIDATE IS FUNCTIONALLY CORRECT AND THEREFORE EARNS THE MAXIMUM CORRECTNESS-ONLY REWARD. THE FINAL COLUMN NAMES THE REWARD PROPERTY EACH IS DESIGNED TO DEFEAT.

ID	Construction	Defeats
H1	every `#pragma omp` deleted	correctness-only
H2	`parallel for num_threads(1)`	static "has a pragma" checks
H3	loop body wrapped in `critical`	nominal parallelism
H4	one-thread path artificially slowed	self-speedup Σ
H5	better serial algorithm, no threads	reference speedup S, capped
H6	work hoisted outside the timed region	timer-scoped rewards
H7	results cached across repeat runs	repeat-based noise reduction
H8	correct but $O(n)$ oversynchronized	additive rewards with a floor

spanning 12 problem families—sort, scan, dense and sparse linear algebra, search, reduce, histogram, stencil, graph, geometry, Fourier transform, and transform—across seven execution models. We use the OpenMP subset (16 tasks), reusing PAREVAL's function-completion prompts, its randomized-input drivers, and most importantly its serial reference implementations as the trusted answer. Because the reference is a plain sequential program it is immune to the parallel hazards under study, which makes it an honest oracle; because the drivers supply many random inputs, a program cannot pass on one lucky example.

Each prompt states what to compute and gives the function signature, but never says how to parallelize it. The model chooses the synchronization strategy, and therefore the scaling behaviour, on its own.

B. Models: who wrote the code

4 closed and 3 open-weight models generated every program we study, and we chose them so that the degeneracy could not be waved away as an artifact of a weak generator. Four are reached through a single uniform API and belong to one frontier family, which lets us watch the reward behave across a capability jump rather than at a single point: `gpt-5.4`, a current frontier general model, and `gpt-5.2`, the generation before it; `gpt-5.3-codex`, a code-specialized sibling of exactly the kind an HPC team reaches for; and `gpt-5.4-pro`, a higher-reasoning variant that spends more test-time compute per program. If the reward's blind spot were something a stronger or more code-aware model simply grows out of, this panel—two generations, a code specialist, and a reasoning model—is where we would expect to see it close.

It does not. Every model in the panel, the strongest included, emits programs whose measured 8-thread speedup ranges from barely above serial to near-linear and earns full reward for all of them: `gpt-5.4-pro`, our higher-reasoning variant, produces accepted programs spanning $1.9\times$ to $7.4\times$, each scored identically correct. The reward's inability to tell a $2\times$ program from a $7\times$ one is a property of what it measures, not of who is answering. To make that point robust to the specific provider, we add an open-weights panel of 3 models (`gpt-oss`, `minimax-m2`, `qwen3-small`) served on a research cluster, spanning a wide capability gradient. Holding the benchmark, harness, and prompts fixed across all seven, any difference we report is attributable to the reward, not to model style.

C. The frozen sample pool

All reward comparisons in Section VI are computed offline over a single frozen pool: 174 generated programs from 174 runs, each annotated with its compile status and per-configuration correctness verdict, and—for the 63 programs the verifier accepted—wall time at every thread count and repetition. Freezing the pool has three benefits. Every reward formulation is scored on identical samples, so differences are attributable to the reward and not to decoding. The comparison costs no inference budget and can be rerun by anyone. And the pool is releasable in a way that a live API-backed experiment is not.

D. Hardware, compilers, and measurement protocol

Compilation and execution run on a single dedicated multi-core node with no other tenants, using GCC with `-O3 -fopenmp`. We deliberately hold the machine fixed so that every reward comparison is made under identical timing conditions; because efficiency is architecture-relative, we treat cross-machine transfer of the efficiency reward as an explicit limitation (Section VII) rather than a result.

Timing follows a protocol designed to make the efficiency term reproducible enough to be a reward rather than an anecdote:

- Thread counts $n \in \{1, 2, 4, \ldots, 8\}$, with `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. Pinning is reported because unpinned runs are not comparable across repeats.
- 3 repetitions per configuration, each in a *fresh process* with a freshly randomized input, which closes the cross-run caching channel (H7).
- The first repetition is discarded as warm-up; we report the minimum wall time over the remainder.
- The timed region encloses the entire driver call, not a region the generated code controls, which closes the timing-region channel (H6).
- Input sizes chosen large enough to amortize thread-creation overhead, because small inputs systematically hide poor scaling.

E. Correctness verification

Correctness is adjudicated by PAREVAL's driver, extended with a differential sweep: the generated function is compiled against the serial reference, fed identical random inputs, and checked at every thread count and repetition rather than once. A program counts as correct only if it agrees everywhere. Disagreement that varies with thread count, or flickers between repetitions at a fixed thread count, is reported as the signature of a race or an order-dependent bug.

TABLE II
REWARD FORMULATIONS SCORED OVER THE FROZEN SAMPLE POOL.

ID	Form	Source
R1	c	correctness-only (the default)
R2	$c(0.5 + 0.5\,\hat{E})$	ACECode [8] (additive)
R3	$c(0.3 + \hat{E})/1.3$	Kevin [9] (corr.+speedup)
R5	$c \cdot \min(S_p(n)/n, 1)$	**this work** (reference-anchored gate)
R5b	$c \cdot \min(\Sigma_p(n)/n, 1)$	ablation (self-anchored)
R5c	$c \cdot \min(S_p(n)/n, 1) \cdot \min(\Sigma_p(n)/2, 1)$	ablation (decomposed)

$\hat{E} = \min(S_p(n)/n, 1)$; S_p uses the trusted serial reference, Σ_p the program's own one-thread time.

As an independent instrument we cross-check every accepted program with Archer [22], the LLVM happens-before OpenMP race detector built on ThreadSanitizer [23] with OMPT annotations, which reasons about actual synchronization rather than observed output. Where the two instruments agree, the claim rests on more than output differencing.

F. Reward formulations compared

Table II lists the formulations scored over the frozen pool. All are implemented in the released harness; where a published reward has hyperparameters we use the values from the original paper and say so.

VI. RESULTS

We report six results. The first two establish that the degeneracy is real and costly. The third shows it also destroys the training gradient. The fourth and fifth compare reward formulations, including where ours fails. The sixth establishes that the performance measurement is reliable enough to be used as a reward at all, which is the precondition for everything before it.

All proportions are reported with Wilson 95% confidence intervals.

A. The serial projection preserves the reward exactly

We applied the serial projection π_{serial} (Definition 1) to each of the 63 programs the differential verifier accepted, and re-ran the full correctness harness on the result.

61 of 63 (96.8%) stripped programs remain correct. Their mean change in correctness-only reward is 0.00, which is exactly zero by construction and which we report only because it is the point: the reward cannot distinguish these programs from the ones they were derived from. Their mean gated reward is 0.125, against -0.71 for the originals.

Figure 2 shows the paired shift. Under R_{corr} every point lies on the diagonal. Under R_{gate} every point drops to the serial floor. This is Proposition 1, executed rather than argued.

B. What the reward cannot see

Holding correctness fixed, we measured the second axis directly: every accepted program was compiled against

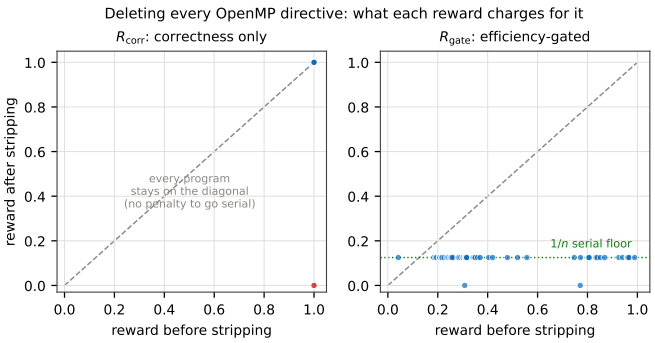

Fig. 2. The degeneracy, measured. Each point is one accepted program, before (x) and after (y) deleting every OpenMP directive. Left: correctness-only reward—all points on the diagonal, no program loses anything. Right: the gated reward—every point collapses to the $1/n$ floor.

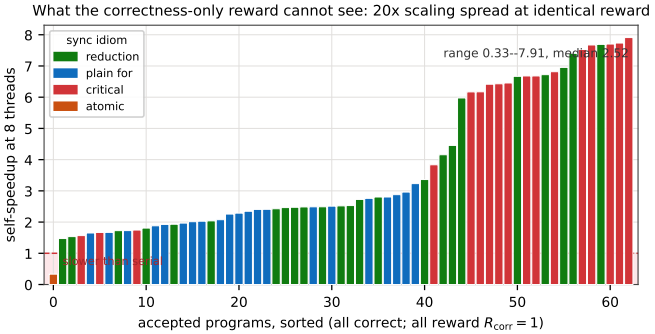

Fig. 3. Sorted 8-thread self-speedup for all 63 accepted programs, coloured by synchronization idiom; the shaded band marks programs slower than serial. Every bar is a program the correctness-only reward scores identically at $R_{\text{corr}} = 1$, yet the values span 0.33× to 7.91×. On a single task alone (histogram) two equally-certified programs range from 0.33–6.82×.

PAREVAL's timing driver at a size large enough to amortize thread overhead and timed across the sweep.

Measured 8-thread speedup among identically-rewarded programs spans 0.33× to 7.91×, with a median of 2.52×. 23.8% of accepted programs achieve capped efficiency below 0.25. 23.8% fail to reach 2× on eight cores. Most starkly, 1.6% run *slower* on eight threads than on one: correct, certified, released by the agent, and worse than the sequential code they were written to replace. A serial program, we reiterate, would have been accepted with the identical score.

The widest within-task spread is 0.33–6.82, on programs the verifier certified as equally robust for the same task (Figure 3). Correctness cannot tell them apart. Only the performance axis can.

We are deliberately careful about mechanism. The synchronization idiom a model chose does not cleanly predict poor scaling—programs using `critical` frequently used it for a cheap $O(\text{threads})$ merge of per-thread partials and scaled well—so we make no claim that models systematically over-synchronize. The defensible claim is narrower and stronger for being exact: among programs an output-only reward rates identically, measured scaling varies by a factor of roughly 20

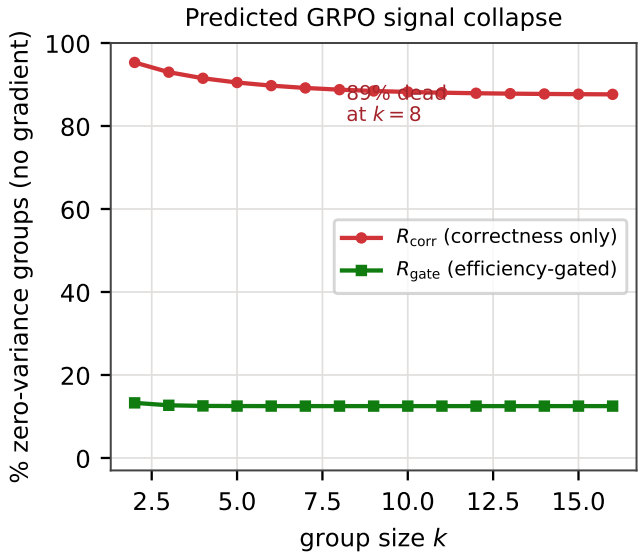

Fig. 4. Fraction of sampled groups carrying zero advantage, versus group size k, for each reward. The correctness-only reward loses signal as capability rises; a reward with a continuous performance term does not.

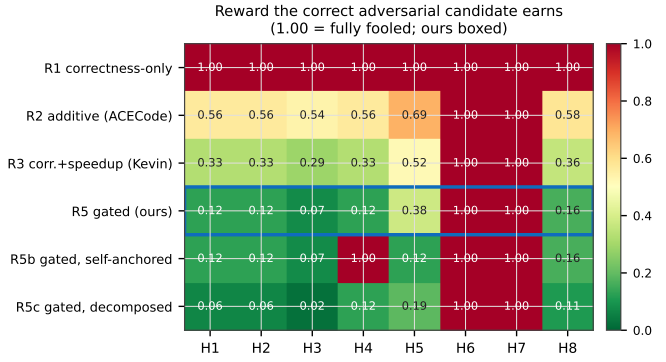

Fig. 5. Reward × hack matrix over PARALLELGATE: the reward paid to a *correct* adversarial candidate of each class (cell value; red = fooled, 1.00 is total failure). Correctness-only (R1) is red across the board; the additive reward (R2) keeps a visible floor (≥ 0.54) even for serial code; our reference-anchored gate (R5, boxed) holds every reward-*shape* attack near the $1/n$ serial floor and moves only under H6/H7, which corrupt the timer itself. R5b's red H4 cell is the price of self-anchoring; R5c is uniformly darkest, including on H5.

(0.33–6.82× on a single task).

C. Signal collapse under group-relative estimators

Proposition 2 predicts that a saturated correctness distribution destroys the group-relative gradient. We instantiate that prediction from the *measured* per-task pass rates: for each task, Equation (7) gives the probability that a group of k i.i.d. samples is dead, and we average it over the 16 tasks. Because our study draws one sample per model–condition cell rather than k samples per prompt, this is the dead-group rate implied by the observed pass rates, not a resampling of a k-sample pool—a falsifiable prediction a training run can check against.

Pooled single-shot correctness is 82.8%, and 75.0% of tasks have a per-task pass rate of exactly 1—for those tasks, *every* group is dead under R_{corr} regardless of k. Overall, the dead-group fraction under R_{corr} is 91.5% at $k = 4$, 88.8% at $k = 8$, and 87.6% at $k = 16$. Under R_{gate}, whose continuous efficiency term rarely ties across correct programs, a group dies only when all k samples are *incorrect*, so the figures fall to 12.5%, 12.5%, and 12.5%—the residual being the few tasks no sample solves, where no reward can help and which does not grow with k.

The reading is that a correctness-only objective is self-limiting: the better the policy gets, the less it can learn. This is not a claim about our particular models—it follows from Equation (7) for any $p_t \to 1$—but it is worth having measured, because the magnitude at realistic pass rates is larger than intuition suggests.

D. Reward bake-off

Treating the accepted programs for a task as the candidates a best-of-N or group-relative selection chooses among, we asked which program each reward selects. R_{corr} is indifferent—every accepted program ties at 1, so an uninformed selection realizes the pool mean in expectation.

Under matched selection over identical samples, the correctness-only reward realizes a mean 8-thread speedup of 3.73× against the gated reward's 4.28× (15.0% higher), with the oracle at 4.28×. The two rewards select a different program in 12 of the tasks with more than one accepted candidate; wherever they differ, the correctness-only choice leaves measured speedup on the table that the gated choice recovers at no cost in correctness.

We label this analysis as a proxy. Best-of-N captures the selection component of group-relative optimization but not its distribution-sharpening effect [15], [16]; a policy trained under R_{corr} would additionally concentrate mass, which our static analysis cannot model. We therefore read these numbers as a lower bound on the divergence between the two objectives, not as a simulation of training.

E. The hack suite: where each reward breaks

Figure 5 scores every reward formulation against every PARALLELGATE class. Each candidate is functionally correct, so R1 awards all of them the maximum—100% by construction. This is the compact statement of the paper: a reward that accepts every entry in an adversarial suite designed to strip out parallelism is not measuring parallelism.

The gate resists 75% of the suite. It does not resist all of it, and the failures are the ones Section IV-C predicted: H6, H7. The self-anchored ablation R5b is defeated by H4 as expected, confirming that anchoring to the fixed serial reference rather than to the program's own one-thread time is not a stylistic choice. The decomposed variant R5c is the only formulation that separates algorithmic from parallel speedup and therefore the only one that resists H5, at the cost of requiring two measurements instead of one.

We report our own failures deliberately. A reward-hack suite whose author's reward passes everything is a suite that was fitted to the reward.

F. Measurement reliability

An efficiency reward is only a reward if the timing is repeatable and if the attack surface of the measurement is closed. Our protocol (Section V-D) times each configuration in a fresh process with freshly randomized input and reports the minimum wall time over 3 repetitions after a discarded warm-up, which closes the cross-run caching channel (H7); the timed region encloses the entire driver call, closing the timing-region channel (H6). What the reward reads is invariant to correctness but not to performance: as Observation 1 requires, deleting or reshaping directives cannot change the computed output, only the wall time—which is precisely why an output-equivalence gate is blind to it. Because efficiency is measured on one machine it is a quantity relative to that machine; we do not claim cross-architecture transfer and flag it as a limitation (Section VII).

G. Correctness is saturated—the premise, not the finding

Finally we report the correctness measurements that motivated the whole inquiry, and we report them briefly because they are the setup rather than the conclusion.

Single-shot generation is robustly correct 82.8% of the time (95% CI [71.1, 90.4]). Adding a repair loop driven by an execute-once tool raises this to 87.9% ([77.1, 94.0]). Replacing that tool with a full thread-sweep differential verifier yields 87.9% ([77.1, 94.0]), a difference of +0.0%. Across 58 paired task–model cells the two tools reached a different final verdict in 0 cases. Excluding a specification ambiguity we identified by hand-inspection on the scan family—where every model computed a defensible reverse-indexed suffix sum against a reference that scans the reversed input—robust correctness is 94.4%; we report this back as a specification ambiguity in the benchmark, not as a defect in the models.

Archer found 0 data races in the 63 accepted programs, while correctly flagging the injected controls and the 1 genuine race shipped by the smallest open-weights model. Two independent instruments—output-differential and happens-before—agree that the accepted set is race-free.

The honest reading is that correctness verification remains the right first gate and that frontier models have made its cheapest form sufficient on idiomatic tasks. That is precisely why the reward cannot stop there.

VII. Discussion

A. What this changes for people building HPC coding agents

Three recommendations follow directly from the measurements, and none of them require adopting our reward.

Do not use output equivalence as an acceptance gate for parallel code. It is necessary and it is not sufficient, and the gap is not small: in our accepted set it is the difference between 0.33× and 7.91×. If an agent's loop terminates when the output matches, it will terminate on programs that are slower than the code they replaced.

Report the target machine alongside any performance reward. Correctness is portable; efficiency is not. A model post-trained against measured speedup on one architecture has been aligned to that architecture. We measure on a single machine and do not claim cross-architecture transfer; characterizing how far the efficiency ranking of these programs moves across machines is the natural next experiment, and one the released harness is built to run.

Report reward resolution. If the noise floor of the timing harness exceeds the differences the reward is meant to express, the reward is noise. This is cheap to measure and, as far as we can tell, not currently reported by anyone—ourselves included, until we looked.

B. Why we do not claim a trained policy

The natural completion of this work is to post-train under both rewards and show that the correctness-only arm drifts toward correct-but-serial code while the gated arm does not. We deliberately do not report such a run.

The reason is that we would rather state a falsifiable prediction than a rushed training curve. The prediction is specific: under GRPO with R_{corr} on a task distribution with p_t near saturation, mean sampled efficiency should be flat or declining while correctness stays constant, because Proposition 2 says most groups contribute no gradient and Proposition 1 says the surviving gradient does not point toward parallelism. Under R_{gate} on the same distribution, correctness should be preserved and mean sampled efficiency should rise. The harness we release computes both rewards over sampled completions and is wired for this experiment; we state the prediction so that it can be checked against us.

C. Threats to validity

The proposition may look trivial: It is elementary once stated, and we say so in Section III. Our defence is that it is not stated anywhere in the RLVR-for-HPC literature we surveyed, that correctness-only verifiable rewards are being deployed for this exact task, and that the corollary about signal collapse (Proposition 2) and the measured cost (Section VI-D) are not elementary.

Prior art on the phenomenon: PARACODEX [7] documented correctness-passing code that executes serially, in a GPU-offload setting, in January 2026. We do not claim priority on the observation. We claim the reframing—from an agent and harness defect to be patched, to the location of the reward function's optimum—and the CPU thread-scaling instantiation.

Benchmark difficulty: A reviewer may object that correctness looks saturated only because PAREVAL's OpenMP tasks are idiomatic. We agree, and it is the argument: on tasks where correctness saturates, a correctness-only reward has nothing left to teach, while the efficiency axis remains wide open. The objection would bite if the efficiency spread narrowed on easy tasks. It does not—the 0.33×–7.91× range is measured on exactly these saturated tasks.

Best-of-N is not GRPO: Our selection analysis captures the selection component of group-relative optimization and not its distribution-sharpening effect [15], [16]. We label it as a proxy wherever it appears and read it as a lower bound.

Scope: Shared-memory OpenMP, 16 PARALLELGATE tasks, a single measurement machine, one serial reference per task. We do not generalize to MPI, CUDA, or Kokkos, although the argument in Section III does not depend on the programming model and the harness supports them. The hazard list underlying PARALLELGATE is not exhaustive, and we expect it to be extended—that is the point of releasing it.

The reference defines the task: Each task has a single serial reference. Being sequential it is immune to the bugs under study, but it also fixes what the task means; the scan-family ambiguity in Section VI-G is a concrete instance of that fixing being contestable.

VIII. RELATED WORK

A. Verifiable rewards for code, with and without performance

Post-training code models with reinforcement learning from verifiable rewards—where an execution check rather than a human supplies the signal [1], [2], optimized with GRPO [3] or its successors [17]–[19]—is now standard. Several efforts include a performance term. ACECode [8] fine-tunes with a dual-objective rewarder that adds an efficiency component to a correctness base for Python runtime. Kevin [9] rewards correctness plus a speedup ratio for GPU kernels and documents the model lazily copying the reference implementation rather than writing one. CUDA-L1 [10] rewards speedup and reports timing exploits in which extra asynchronous streams mislead the timer. CUDAPerf [11], CUDA Agent [24], and MaxCode [12] similarly fuse correctness with measured speedup.

Two lines are closest to ours and both come from the HPC community. RLPF [13] aligns code models to performance using a learned reward model trained on fast/slow code pairs, with reported OpenMP gains of 1.9–4.5×; the reward is predicted rather than measured, which is a different design point with different failure modes, but the objective—get the model to write faster parallel code—is the same one we are arguing for. Real-machine reward GRPO [14] post-trains an HPC code model using measured GFLOPS on a supercomputer behind a staged penalty ladder, and is the nearest existing instance of an execution-measured HPC reward. We differ in object rather than in ambition: that work optimizes throughput on a kernel; we ask why the correctness term cannot stand alone, and what structure follows.

Our contribution relative to all of the above is not the combination of correctness and performance. It is (i) the argument that the correctness term's optimum contains the serial program, which is what motivates a multiplicative gate rather than a weighted sum, and (ii) an adversarial suite that turns "which reward is better" into a measurable question.

B. Reward hacking and verifier gaming

That verifiable rewards can be gamed is well documented. Recent work distinguishes extensional from intensional verification and shows that verifiers checking only extensional correctness admit false positives [21]; others fuzz RLVR verifiers directly [25] and analyze reward hacking in rubric-based RL [26]. Our serial projection is an extensional-verification false positive of a particularly clean kind: it is not an exploit found by search but a transform guaranteed to succeed on every correct program. PARALLELGATE follows this literature's methodology—attack the reward, report what survives—and instantiates it for parallel code.

C. LLMs for parallel and HPC code

PAREVAL [4] is the standard benchmark, evaluating pass@k, $\text{speedup}_n@k$, and $\text{efficiency}_n@k$ for model-generated OpenMP, MPI, CUDA, and Kokkos, and documenting a large gap between models' serial and parallel competence. Our capped efficiency (Equation (8)) is that benchmark's efficiency metric with a unit cap, and we adopt it deliberately rather than inventing a variant. PAREVAL-Repo [27] extends the setting to repository-level translation. Model lines specialized for HPC include HPC-Coder [28], HPC-Coder-V2 [29], and OMPGPT [30], and PCEBench [31] evaluates parallel code on both correctness and performance. Work on whether models can predict parallel code performance at all [32] reinforces the premise that performance is not implied by correctness.

Agentic HPC systems have proliferated: LASSI [33] builds a compile-and-run self-correction loop for parallel-code translation, MARCO [34] and VibeCodeHPC [35] apply multi-agent and auto-tuning approaches, UniPar [36] targets parallelization, and ParEVO [37] guides an evolutionary loop with correctness verification, race detection, and performance profiling—the closest existing system to a two-signal loop, though the signals guide search rather than serve as a reward.

D. Functional-correctness and efficiency evaluation

Code-generation evaluation has centered on functional correctness of small self-contained problems [38] and, at repository scale, on SWE-bench [39]. EvalPlus [40] showed that code passing a visible test suite often fails under additional tests; our result is a different failure of the same shape, along an axis no additional test can reach. Efficiency-aware benchmarks—EffiBench [41], ENAMEL [42] with its right-tail $\text{eff}@k$ estimator, and KernelBench [43] for GPU kernels—establish that measured performance belongs in evaluation. We argue it belongs in the reward.

E. Dynamic race detection

A mature line of HPC tooling detects data races dynamically: ThreadSanitizer [23] instruments memory accesses at scale, and Archer [22], ROMP [44], SWORD [45], and OMPRacer [46] specialize happens-before detection for OpenMP. These are far more powerful race detectors than output differencing, and we use Archer as an independent instrument rather than as a competitor. Our finding is not a better detector: it is that for frontier-model output on idiomatic tasks the race they hunt is largely absent, and the binding constraint has moved to whether the accepted code is parallel at all.

F. Differential and metamorphic testing

Executing a program under multiple configurations and flagging disagreements is the essence of differential testing [47]; checking properties preserved under transformation is metamorphic testing [48]. The serial projection is, in this vocabulary, a metamorphic relation run in reverse: instead of a transform under which the output should be preserved and a violation indicates a bug, it is a transform under which the output *is* preserved and that preservation indicates a defect in the reward.

IX. Conclusion

Agentic AI will help HPC only as much as we can trust what it writes once that code runs in parallel, and what we trust is whatever the checker says. So the checker is the thing to get right—first because an agent accepts its own work on it, and increasingly because it is the reward a post-training loop optimizes.

We showed that the natural checker for parallel code cannot serve as that reward. Delete every OpenMP directive from a correct program and it stays correct, so the serial program sits in the reward's argmax and the reward is invariant to the property the task exists to obtain. We made this executable: 96.8% of the programs our verifier accepted survive mechanical pragma-stripping at exactly zero reward cost. We measured what the reward cannot see: accepted programs span $0.33\times$ to $7.91\times$ on eight threads, 1.6% of them slower in parallel than serially, all scored identically. And we showed the failure is not only at the optimum but at the gradient: with correctness saturated at 82.8%, group-relative estimators see zero reward variance on 88.8% of sampled groups and learn nothing at all.

The fix that follows from the argument is structural rather than additive. Gating a capped strong-scaling efficiency behind correctness removes the serial program from the optimum, restores the gradient, and—unlike a weighted sum—makes it impossible for chasing reward to buy a wrong answer. Under matched selection it recovers 15.0% more speedup than a correctness-only objective from the same samples. It is not immune to attack, and we say which of our own 48 adversarial candidates defeat it and why.

We release ParallelGate—the adversarial suite, the reward implementations, the frozen sample pool with timings, the prompts, and the transcripts—so that the reward comparisons in this paper can be recomputed without an inference budget, and so that the next reward design for HPC code can be tested rather than proposed. Correctness stopped being the binding constraint. Correct-but-not-parallel is where this code now fails, and a reward that looks only at the output cannot see it.

Acknowledgments

References

[1] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda, A. Liu, N. Dziri, S. Lyu, Y. Gu, S. Malik, V. Graf, J. D. Hwang, J. Yang, R. Le Bras, O. Tafjord, C. Wilhelm, L. Soldaini, N. A. Smith, Y. Wang, P. Dasigi, and H. Hajishirzi, "TÜLU 3: Pushing frontiers in open language model post-training," *arXiv preprint arXiv:2411.15124*, 2024.

[2] D. Guo, D. Yang, H. Zhang, J. Song *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, 2025, arXiv:2501.12948.

[3] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[4] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?" in *Proc. 33rd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC)*, 2024, pp. 281–294, arXiv:2401.12554.

[5] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Int. Conf. Learning Representations (ICLR)*, 2024.

[6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] E. Kaplan, T. Bitan, L. Ghrayeb, L. Chen, T. Yotam, N. Hasabnis, and G. Oren, "ParaCodex: A profiling-guided autonomous coding agent for reliable parallel code generation and translation," in *Proc. 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 16 113–16 136, arXiv:2601.04327.

[8] C. Yang, H. J. Kang, J. Shi, and D. Lo, "ACECode: A reinforcement learning framework for aligning code efficiency and correctness in code language models," *arXiv preprint arXiv:2412.17264*, 2024.

[9] C. Baronio, P. Marsella, B. Pan, S. Guo, and S. Alberti, "Kevin: Multi-turn RL for generating CUDA kernels," *arXiv preprint arXiv:2507.11948*, 2025.

[10] X. Li, X. Sun, A. Wang, J. Li, and C. Shum, "CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning," *arXiv preprint arXiv:2507.14111*, 2025.

[11] Q. I. Mahmud, N. K. Ahmed, and A. Jannesari, "Multi-turn RL with structural and performance aware rewards for CUDA kernel generation," *arXiv preprint arXiv:2607.20908*, 2026.

[12] J. Ou, S. Chaudhary, K. Bostrom, N. Weir, S. Zhang, H. Rangwala, and G. Karypis, "MaxCode: A max-reward reinforcement learning framework for automated code optimization," *arXiv preprint arXiv:2601.05475*, 2026.

[13] D. Nichols, P. Polasam, H. Menon, A. Marathe, T. Gamblin, and A. Bhatele, "Performance-aligned LLMs for generating fast code," *arXiv preprint arXiv:2404.18864*, 2024.

[14] R. Mikasa, S.-i. Hayashi, D. Mukunoki, T. Hoshino, and T. Katagiri, "Improving HPC code generation capability of LLMs via online reinforcement learning with real-machine benchmark rewards," *arXiv preprint arXiv:2602.12049*, 2026.

[15] F. Bagirov, M. Arkhipov, K. Sycheva, E. Glukhov, and E. Bogomolov, "The best of N worlds: Aligning reinforcement learning with best-of-N sampling via max@k optimisation," *arXiv preprint arXiv:2510.23393*, 2025.

[16] A. W. He, D. Fried, and S. Welleck, "Rewarding the unlikely: Lifting GRPO beyond distribution sharpening," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 25 548–25 560, arXiv:2506.02355.

[17] Q. Yu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025.

[18] Z. Liu *et al.*, "Understanding R1-zero-like training: A critical perspective," *arXiv preprint arXiv:2503.20783*, 2025.

[19] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, J. Zhou, and J. Lin, "Group sequence policy optimization," *arXiv preprint arXiv:2507.18071*, 2025.

[20] Y. Y. Bay and K. A. Yearick, "GRPO, Dr. GRPO, and DAPO are three operations on one number: The group-standard-deviation identity," *arXiv preprint arXiv:2607.00152*, 2026.

[21] L. Helff, Q. Delfosse, D. Steinmann, R. Härle, H. Shindo, P. Schramowski, W. Stammer, K. Kersting, and F. Friedrich, "LLMs gaming verifiers: RLVR can lead to reward hacking," *arXiv preprint arXiv:2604.15149*, 2026.

[22] S. Atzeni, G. Gopalakrishnan, Z. Rakamaric, D. H. Ahn, I. Laguna, M. Schulz, G. L. Lee, J. Protze, and M. S. Müller, "ARCHER:

Effectively spotting data races in large OpenMP applications," in *IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2016.

[23] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Workshop on Binary Instrumentation and Applications*, 2009.

[24] W. Dai, H. Wu, Q. Yu, H.-a. Gao, J. Li, C. Jiang, W. Lou, Y. Song, H. Yu, J. Chen, W.-Y. Ma, Y.-Q. Zhang, J. Liu, M. Wang, X. Liu, and H. Zhou, "CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation," *arXiv preprint arXiv:2602.24286*, 2026.

[25] J. Ray, "Before the model learns the bug: Fuzzing RLVR verifiers," *arXiv preprint arXiv:2606.01066*, 2026.

[26] X. Wang, Z. Hao, S. Hou, H. Peng, J. Li, and X. Wang, "Reproducing, analyzing, and detecting reward hacking in rubric-based reinforcement learning," *arXiv preprint arXiv:2606.04923*, 2026.

[27] J. H. Davis, D. Nichols, I. Khillan, and A. Bhatele, "ParEval-Repo: A benchmark suite for evaluating LLMs with repository-level HPC translation tasks," in *Proc. 54th Int. Conf. Parallel Processing (ICPP)*, 2025.

[28] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "HPC-Coder: Modeling parallel programs using large language models," in *ISC High Performance 2024 Research Paper Proceedings (39th Int. Conf.)*. IEEE, 2024, pp. 1–12.

[29] A. Chaturvedi, D. Nichols, S. Singh, and A. Bhatele, "HPC-Coder-V2: Studying code LLMs across low-resource parallel languages," in *ISC High Performance*, 2025, arXiv:2412.15178.

[30] L. Chen, A. Bhattacharjee, N. Ahmed, N. Hasabnis, G. Oren, V. Vo, and A. Jannesari, "OMPGPT: A generative pre-trained transformer model for OpenMP," in *Euro-Par*, 2024.

[31] L. Chen, N. K. Ahmed, M. Capotă, T. Willke, N. Hasabnis, and A. Jannesari, "PCEBench: A multi-dimensional benchmark for evaluating large language models in parallel code generation," in *IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2025, pp. 546–557.

[32] G. Bolet, G. Georgakoudis, H. Menon, K. Parasyris, N. Hasabnis, H. Estes, K. W. Cameron, and G. Oren, "Can LLMs predict parallel code performance?" in *Proc. 34th Int. Symp. High-Performance Parallel and Distributed Computing (HPDC)*, 2025, arXiv:2505.03988.

[33] M. T. Dearing, Y. Tao, X. Wu, Z. Lan, and V. Taylor, "LASSI: An LLM-based automated self-correcting pipeline for translating parallel scientific codes," in *2024 IEEE Int. Conf. Cluster Computing Workshops (CLUSTER Workshops)*, 2024, pp. 136–143, arXiv:2407.01638.

[34] A. Rahman, V. Cvetkovic, K. Reece, A. Walters, Y. Hassan, A. Tummeti, B. Torres, D. Cooney, M. Ellis, and D. S. Nikolopoulos, "MARCO: Multi-agent code optimization with real-time knowledge integration for high-performance computing," *arXiv preprint arXiv:2505.03906*, 2025.

[35] S.-i. Hayashi, K. Morita, D. Mukunoki, T. Hoshino, and T. Katagiri, "VibeCodeHPC: An agent-based iterative prompting auto-tuner for HPC code generation using LLMs," *arXiv preprint arXiv:2510.00031*, 2025.

[36] T. Bitan, T. Kadosh, E. Kaplan, S. Meiri, L. Chen, P. Morales, N. Hasabnis, and G. Oren, "UniPar: A unified LLM-based framework for parallel and accelerated code translation in HPC," in *2025 IEEE High Performance Extreme Computing Conf. (HPEC)*, 2025, pp. 1–9, arXiv:2509.12136.

[37] L. Yang, Z. Nie, A. Liu, F. Zou, D. Altinbüken, A. Yazdanbakhsh, and Q. C. Liu, "ParEVO: Synthesizing code for irregular data: High-performance parallelism through agentic evolution," *arXiv preprint arXiv:2603.02510*, 2026.

[38] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[39] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Int. Conf. Learning Representations (ICLR)*, 2024.

[40] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.01210.

[41] D. Huang, Y. Qing, W. Shang, H. Cui, and J. M. Zhang, "EffiBench: Benchmarking the efficiency of automatically generated code," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2402.02037.

[42] R. Qiu, W. W. Zeng, J. Ezick, C. Lott, and H. Tong, "How efficient is LLM-generated code? a rigorous and high-standard benchmark," in *Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2406.06647.

[43] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" in *Int. Conf. Machine Learning (ICML)*, 2025, arXiv:2502.10517.

[44] Y. Gu and J. Mellor-Crummey, "Dynamic data race detection for OpenMP programs," in *Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2018.

[45] S. Atzeni, G. Gopalakrishnan, Z. Rakamaric, I. Laguna, G. L. Lee, and D. H. Ahn, "SWORD: A bounded memory-overhead detector of OpenMP data races in production runs," in *IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2018.

[46] B. Swain, Y. Li, P. Liu, I. Laguna, G. Georgakoudis, and J. Huang, "OMPRacer: A scalable and precise static race detector for OpenMP programs," in *Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[47] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[48] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, 2018.

ARTIFACT DESCRIPTION

Summary of the experiments reported

The paper reports six experiment groups: (E1) serial-projection preservation, (E2) efficiency spread across accepted programs, (E3) dead-group fraction under group-relative estimators, (E4) reward bake-off by matched selection, (E5) the PARALLELGATE hack matrix, and (E6) the measurement protocol on a single machine.

Crucially, **E1–E5 require no model inference and no multi-core node.** They are computed offline over a frozen pool of 174 generated programs released with the artifact, each annotated with compile status, per-configuration correctness, and wall time at every thread count. A reviewer reproduces every reward comparison, every figure, and the entire grounded diagnostician on a laptop in minutes; the timings themselves (E2/E6) were produced once on a multi-core node and are shipped as data.

Artifact availability

Repository and a tagged, DOI-archived release are provided; the review copy is anonymized. The code is released under a permissive open-source license, and the frozen pool—generations, verdicts, and timings—is included so that no API access is required to recompute the paper.

Software dependencies

- Compiler: GCC with `-O3 -fopenmp`
- Race detection: LLVM Archer (ThreadSanitizer + OMPT)
- Python 3.12; dependencies pinned in `requirements.txt` (the grounded diagnostician additionally uses the `openai` client with Entra ID bearer-token auth)

Hardware

Timings were collected on a single dedicated multi-core x86-64 node with threads pinned (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and no other tenants. Reproducing E1–E5 requires no special hardware.

Repository layout

- `agent/diag_core.py` — the grounded evidence layer over the frozen pool (Section II)
- `agent/hpc_agent.py` — the tool-calling diagnostician and the performance-critic
- `agent/parallel_gate.py` — PARALLELGATE scoring; produces Figure 5
- `agent/compute_numbers.py` — recomputes E1–E4; writes the paper's `numbers` macros
- `agent/make_paper_figs.py` — regenerates Figures 2, 3, 4
- `results/` — the frozen pool: `scaling.jsonl` (per-thread timings), `pareval_runs.jsonl` (verdicts), `sanitizer.jsonl`, `oversync.jsonl`
- `logs/pareval/generations/` — every generated program with its prompt and transcript

Reproduction steps

1) `python -m agent.compute_numbers --write` — recompute E1–E4 offline; writes `numbers_filled.tex`.
2) `python -m agent.parallel_gate --write` — score PARALLELGATE; writes `hacks_numbers.tex` and `table_hacks.tex`.
3) `python -m agent.make_paper_figs` — regenerate all figures.
4) `pdflatex main` — rebuild the PDF. **Every number comes from a generated macro file; there are no literal numbers in the section sources.**

Expected variation

E1–E5 are deterministic given the frozen pool and reproduce exactly. The raw timings underlying E2 are machine-dependent: absolute speedups will differ on other hardware, but the *ordering* claims and the serial-projection result should not.